

```
>> [centers, radii] = imfindcircles(RGB,
[40 60], 'ObjectPolarity', 'Dark',
'Sensitivity', 0.9)
```

You will get:

```
centers =
384.2002 204.0138
248.6127 201.3416
198.2825 73.5618
442.8091 77.9060
radii =
50.1646
50.2572
50.2951
49.7827
```

By increasing sensitivity to 0.9, the function `imfindcircles` found four circles. 'centers' contains center locations of circles and 'radii' contains the estimated radii of these circles.

7. Function `viscircles` can be used to draw circles on the image. Output variables 'centers' and 'radii' from function `imfindcircles` can be passed directly to function `viscircles`:

```
>> imshow(RGB)
>> h = viscircles(centers, radii)
```

Fig. 13 shows the result.

As you can see from Fig. 13, centers of circles seem to be correctly positioned and their corresponding radii seem to match well to the actual circles. However, quite a few circles were still missed. Let us increase 'Sensitivity' to 0.928:

```
>> [centers, radii] = imfindcircles(RGB,
[40 60], 'ObjectPolarity', 'Dark',
'Sensitivity', 0.928)
```

```
centers =
384.2193 204.4776
248.8446 201.4930
197.7027 73.5149
442.9998 78.3674
76.7218 76.5162
radii =
50.1646
50.2572
50.2951
49.7827
50.4039
```

Now, we are able to detect five circles. Hence, by increasing the value of sensitivity you can detect more circles. Let us plot these circles on the image again:

```
>> delete(h); % Delete previously drawn
circles
>> h = viscircles(centers, radii);
```

Fig. 14 shows the result.

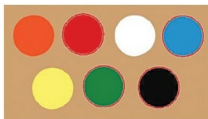


Fig. 13: Circles drawn using function `viscircles`

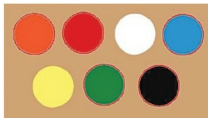


Fig. 14: Replotted circles

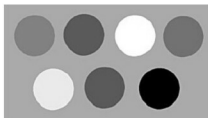


Fig. 15: Grayscale version of the image in Fig. 14

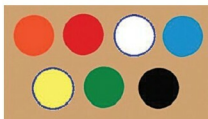


Fig. 16: Bright circles drawn in a different colour

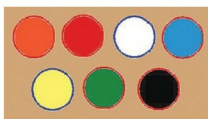


Fig. 17: Both the bright and the dark objects detected and encircled

8. As you can see, function `imfindcircles` does not find yellow and white circles in the image. Yellow and white circles are lighter than the background. In fact, these seem to have very similar intensities as the background. To confirm this, let us see

the grayscale version of the original image again. Fig. 15 shows the grayscale image.

```
>> figure, imshow(gray_image)
```

9. To detect objects brighter than the background, change 'ObjectPolarity' to 'bright':

```
>> [centers, radii] = imfindcircles(RGB,
[40 60], 'ObjectPolarity', 'Bright',
'Sensitivity', 0.98)
```

```
centers =
323.2806 75.9644
122.3421 205.1504
radii =
49.7654
50.1574
```

10. Draw bright circles in a different colour by changing 'Colour' parameter in `viscircles`:

```
>> imshow(RGB)
>> hBright = viscircles(centers, radii,
'Color', 'b');
```

Fig. 16 shows the image.

11. There is another parameter in function `imfindcircles`, namely, `EdgeThreshold`, which controls how high the gradient value at a pixel has to be before it is considered an edge pixel and included in computation. A high value (closer to 1) for this parameter will allow only the strong edges (higher gradient values) to be included, whereas a low value (closer to 0) is more permissive and includes even the weaker edges (lower gradient values) in computation. Therefore, lower the value of `EdgeThreshold` parameter, more are the chances of a circle's detection. However, it also increases the likelihood of detecting false circles. Hence, there is a trade-off between the number of true circles that can be found (detection rate) and the number of false circles that are found with them (false alarm rate).

Following commands detect both the bright and the dark objects and encircle them:

```
>> [centersBright, radiiBright, metricBright] = imfindcircles(RGB, [40 60], 'ObjectPolarity',
'bright', 'Sensitivity', 0.9, 'EdgeThreshold',
0.2);
>> imshow(RGB);
>> h = viscircles(centers, radii);
>> hBright = viscircles(centersBright,
radiiBright, 'Color', 'b');
```

Fig. 17 shows the output image. **EFY**
Concluded